



LANDING ON PLANET **DiSHACLed**

FOR MISSION
survey on commonly used pipelines

Sirus NV

Verlorenbroodstraat 122 Bus 8
9820 Merelbeke
info@sirus.be
+32 9 335 76 43
www.sirus.be

Thomas Carsten

Data Engineer
thomas.carsten@sirus.be



TABLE OF CONTENTS

Introduction.....	3
Objective.....	3
Part 1: Assessing the state-of-the-art.....	4
1.1 Included frameworks	4
1.2 Derived Categories.....	4
1.3 Framework comparison.....	6
1.4 Conclusion.....	7
Part 2: Survey	8
1.5 Design.....	8
1.6 Results	9
1.6.1 Participants.....	9
1.6.2 Example pipelines	9
1.6.3 Step categories.....	9
1.6.4 Pipeline setup and reuse.....	10
1.7 Conclusion.....	11
Conclusion.....	11

INTRODUCTION

Data pipelines are automated workflows that collect data, transform it, and deliver it where it is needed. Many data engineers find that building and maintaining these pipelines takes a lot of time and effort. Even though there are many tools and frameworks for building pipelines, much of the work is still done by hand: custom code is written to clean the data, apply business rules, and bring the data into the right format and structure.

Writing custom code makes it easy to tailor a pipeline to a project, but it also has downsides. It takes time, and the code is often hard to reuse, because it only works in the context it was written for. It may not follow best practices, so it can lack the performance and robustness of mature, well-tested solutions. And because business rules are buried in the code, it is hard to see what a pipeline is meant to do without reading the code or extra documentation. This becomes a problem when someone else takes over the pipeline.

The DiSHACLeD project aims to show a different way of building data pipelines that solves these problems. **The idea is to build pipelines from modular transformation steps, where each step does one well-defined operation and can be configured for a specific use case. The pipelines and their steps are described semantically, that is, as Linked Data, and a pipeline is run by interpreting this description. This separates what a pipeline should do (its intent) from how it is done (its implementation).** We call this approach the "Generic Toolchain Architecture".

This approach has several advantages:

- Readability: a semantic description gives a clear overview of what a pipeline does, and makes steps and pipelines easier to find and share.
- Efficiency: each step only has to be built once. Configuration is enough to adapt it to a new use case.
- No lock-in: because steps are run by interpreting their description, the same step can be implemented in different frameworks, and one pipeline can combine steps from several frameworks.
- Linked Data benefits: pipeline descriptions can be validated before they are run, suitable steps can be found by querying, and datasets can be linked to the pipelines that produced them, which gives full provenance.

OBJECTIVE

With this survey, we want to understand how data pipelines are built today. In particular, we want to know which transformation steps and pipeline patterns are most common, especially in the Linked Data space. This tells us where reusable steps would be most valuable, and whether the Generic Toolchain Architecture can cover the pipelines people actually build.

The survey consisted of two parts:

- In Part 1, we looked at existing open-source frameworks that build pipelines from modular steps and can transform RDF data. We collected the names and descriptions of all their transformation steps and grouped them into categories. We also assessed how well each framework could serve as a foundation for the Generic Toolchain Architecture.
- In Part 2, we ran an online survey in which participants drew an example pipeline from their daily work in an interactive pipeline builder. We then mapped the steps they described to the categories from Part 1, to see which steps are most common and in what order they appear.

PART 1: ASSESSING THE STATE-OF-THE-ART

1.1 INCLUDED FRAMEWORKS

To be included, a framework had to meet three requirements. It had to:

- consist of modular transformation steps that can be chained in different ways
- be able to transform RDF data
- be fully open source

Ten frameworks met these requirements (Table 1).

Table 1.

framework	number of scraped items
Apache NiFi	290
KNIME	273
semantic.works	259
LinkedPipes ETL	77
barnard59	60
RDF-Connect	31
RDFpro	20
LDIO	18
Fintan	16
Chimera	11
	1055

1.2 DERIVED CATEGORIES

For each of the 1055 transformation steps in these frameworks, we collected its name, its description and, where available, the category the framework itself gave it. We then rewrote each step in the same short format: an action verb, followed by a preposition and an object, such as "write to dataset" or "validate against schema". Writing all steps in the same way made it easy to spot common themes. This led to 15 categories, grouped into four parent categories: In/Out, Refinement, Insight, and Pipeline Execution. Table 2 gives an overview of the categories with example steps.

Table 2

parent category	category		
In/Out	Read Data	In	description action verbs examples Processors that load data into the pipeline for further processing. <i>capture, consume, extract, fetch, get, import, ingest, load, open, parse, pull, read, retrieve</i> open a file to load its contents into memory, receive data packets from a messaging system or queue
	Write Out Data		description action verbs examples Processors that export data from the pipeline for storage or receipt. <i>dump, insert, materialize, package, post, publish, push, put, send, stream, write</i> transfer data from a source system to a target location, save data onto a physical storage
	File Handling		description action verbs examples Processors that commit changes on the storage structure of the files holding the data. <i>copy, create, delete, find, list, move, purge, remove, rename</i> move file from one location to another, create a duplicate of a file

Refinement	Communication	description	Processors that establish a connection between two systems, allowing the transfer of data.
		action verbs	<i>authenticate, connect, handle, listen, request, sign, verify</i>
		examples	establish a connection between services for data transfer, tap into the flow of continuous data so that it is received when new data arrives
	Data Representation	description	Processors that change how data is represented without changing the content of the data.
		action verbs	<i>compress, convert, encode, encrypt, serialize, unpack, unzip</i>
Refinement		examples	change a file from one format to another, convert the data structure into a predefined format
	Data Selection	description	Processors which select a subset of the data for further processing.
		action verbs	<i>lookup, combine, construct, filter, query, sample, scan, select</i>
		examples	retain only the data entries that meet specific criteria, while excluding others, delete repeated entries from a dataset to ensure each entry is unique
	Data Manipulation	description	Any processor that changes the actual content of the original data in some way.
Refinement		action verbs	<i>anonymize, append, concatenate, deduplicate, enrich, expand, flatten, hash, mark, modify, replace, set, substitute, tag, transform, translate, update</i>
		examples	correct errors and remove inconsistencies in data, correct errors and remove inconsistencies in data
	Data Validation	description	Processors that check whether the incoming data fulfils predefined criteria.
		action verbs	<i>ask, validate</i>
		examples	check whether data conforms to a predefined structure, verify whether all required data is present in datasets

Parent category	category		
Insight	Data Summarization	description	Processors that calculate metrics for the data to describe it.
		action verbs examples	<i>calculate, compute, count, describe, score</i> find and count how often certain data entries appear within a dataset, calculate a metric that describes the data in some way
	Data Inference	description	Processors aimed at making predictions or drawing conclusions based on the data.
		action verbs examples	<i>analyze, classify, cluster, comprehend, estimate, identify, predict, test</i> perform a statistical test to evaluate whether patterns in data are due to chance, assign items to categories based on its characteristics
	Data Visualization	description action verbs examples	Processors which visualize the data for inspection. <i>explore, inspect, plot, view</i> visualize data with a graphic to reveal patterns, trends and relationships, Show distributions, outliers, and missing values to assess data quality
Pipeline Execution	Data Learning	description action verbs examples	Processors which train a model based on the data. <i>embed, finetune, initialize, label, layer, learn, pool</i> teach a machine learning algorithm to recognize patterns in data, retrain a model with task-specific data to improve its performance on that task
	Flow Control	description action verbs	Processors that control the execution of the pipeline. <i>assign, batch, buffer, control, detect, distribute, end, enforce, fork, join, limit, merge, monitor, notify, partition, retry, route, segment, split, start, wait</i>
		examples	track the operations and performance of a data pipeline, spread workload across systems to optimize performance
	Execute Process	description	Processors which allow the execution of a command by taking care of the implementation.
		action verbs examples	<i>apply, compile, execute, invoke, map, prompt, run</i> run a script to have commands executed on data and return its output, start a task which performs operations on data in a predefined way
	Data Generation	description	Processors which generate new data.
		action verbs	<i>debug, generate</i>
		examples	generate mock data for testing or debugging purposes, create rare, outlier, or edge-condition data for robustness testing

1.3 FRAMEWORK COMPARISON

Finally, we assessed how suitable each framework would be as a foundation for the Generic Toolchain Architecture, that is, whether it could be used to build and run a pipeline from any semantic pipeline description. On top of the inclusion requirements, we looked at four capabilities:

- Data streaming: the framework supports both streaming and batch processing. With streaming, records are processed one by one, so they can be passed on to the next step before the whole batch is finished.

- Multi-language support: steps can be implemented in different programming languages, so a wide range of existing implementations can be used.
- Pipeline configuration as a file: the pipeline layout and step settings can be written down in a file. This makes it possible to translate a semantic pipeline description into the framework's own configuration file.
- Branching pipelines: a step can pass its output to zero, one or more following steps, since strictly linear pipelines are of limited use.

Table 3 summarizes the results.

Table 3

framework	data streaming	multi-language support	pipeline configuration as file	branching pipeline
Apache NiFi	✓	✓ Java, Python (since NiFi 2.x)	✓ JSON flow definition, verbose, can be shared and reused	✓
KNIME	● primarily batch-based, streaming only for selected nodes (beta extension)	✓	● workflows are stored as XML files or .knwf archive, but generated by the tool	✓
semantic.works	✗ batch-based, limited by SPARQL queries to the triplestore	✓ any Docker container, templates for JavaScript, Ruby, Python	✓ JSON (job configuration), YAML (docker-compose)	● job configurations are currently linear, but graph-like configurations are possible
LinkedPipes ETL	✗ complete data units are passed between components (chunked processing possible)	✗ JVM	✓ JSON-LD	✓
barnard59	✓	● JavaScript, other languages only via wrappers	✓ RDF	✗ steps form a linear list (sub-pipelines allow merging streams)
RDF-Connect	✓	✓ JavaScript/TypeScript, Python, Java	✓ RDF	✓
RDFpro	✓ some processors require the full dataset	● Java, scripting in Groovy and JavaScript	● pipeline is a command-line expression, no dedicated file format	● parallel branches must be merged again
LDIO	✓	✗ Java	✓ YAML	● linear chain of transformers, but multiple outputs in parallel
Fintan	✓	● Java, other languages via OpenAPI service wrapper	✓ JSON	✓
Chimera	● message-by-message, but each RDF graph is processed as a whole	● Java, Groovy and JavaScript via Apache Camel	✓ Camel routes in XML or YAML	✓ via Apache Camel

1.4 CONCLUSION

In Part 1, we looked at 10 open-source frameworks that transform RDF data by chaining modular steps. The number of steps per framework varied a lot, from 290 for Apache NiFi to 11 for Chimera. By rewriting all 1055 steps in the same format, we could capture them in 15 categories. These cover

reading and writing data, refining it, analysing and learning from it, and controlling how the pipeline runs. The categories give a shared vocabulary for what a step does, regardless of the framework it comes from, and we used them to categorize the pipelines from Part 2.

The framework comparison showed a trade-off. Most frameworks built specifically for Linked Data (LDIO, LinkedPipes ETL, barnard59, RDFpro, Fintan and Chimera) are tied to a single programming language or runtime, usually Java or JavaScript, and support other languages at most through scripting or wrappers. That limits which implementations can be combined in one pipeline. Several of them also only support linear or partly branching pipelines, or process data in batches instead of streaming it. General-purpose frameworks such as Apache NiFi and KNIME support multiple languages and branching, but cannot process RDF out of the box and need extensions or custom processors for it. semantic.works handles RDF natively and supports multiple languages, but is batch-based and its job configurations are currently linear. Only RDF-Connect handles RDF natively and offers all four capabilities: streaming, multiple languages, pipeline configuration as a file (in RDF), and branching. This makes RDF-Connect the most suitable foundation for the Generic Toolchain Architecture at this time.

PART 2: SURVEY

1.5 DESIGN

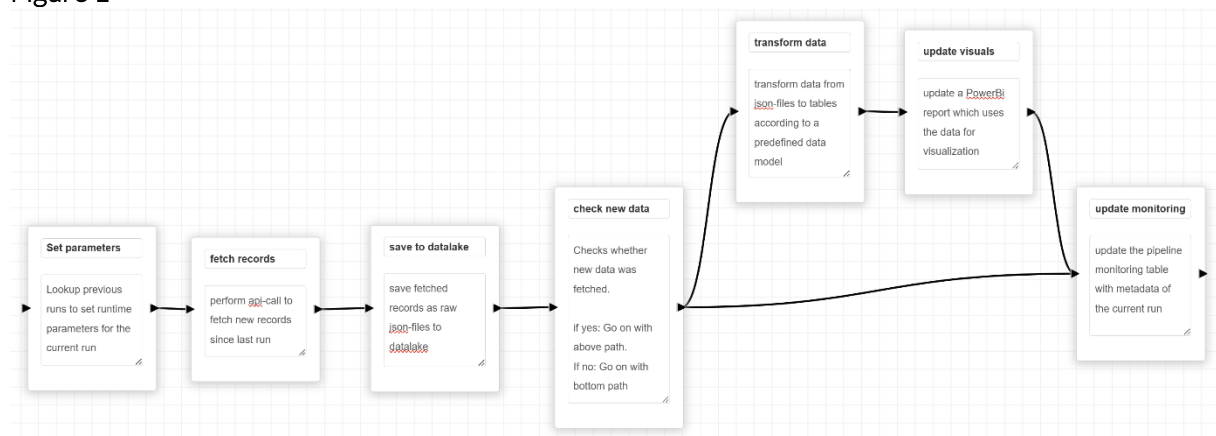
Building on Part 1, we ran an online survey to find out which transformation steps and pipeline patterns people use in practice. The survey was sent to the contacts on the DiSHACled contact list at Sirius, RedPencil, Inuits, imec and Ghent University (UGent).

The survey took around 15 minutes and all responses were anonymous. Participants first gave their job title and years of experience in that role. They were then introduced to the idea of a data pipeline as a series of processing steps, where each step does something with the data and passes it on to the next. Pipelines can be linear or branch out. A short video showed how to use the pipeline builder, after which participants built an example pipeline from their daily work: either a recent pipeline or a representative example. We deliberately did not give them a list of steps to choose from, so as not to steer their answers. Instead, participants added each step themselves, gave it a title and a description in their own words, and linked the steps to show the order in which data flows through the pipeline. This let us map their steps to the Part 1 categories afterwards.

When their pipeline was finished, participants saw it again and were asked to explain it briefly. The survey ended with questions on how many days it takes on average to set up a pipeline for a project, what percentage of their custom data processing logic they can reuse across projects, and which parts of their pipelines cannot be reused. Optionally, participants could leave comments and an email address to receive a copy of this report.

The pipeline builder was made with [Drawflow](#), which lets participants draw a pipeline as a graph: the boxes are transformation steps and the arrows show how data flows between them. The survey itself was built with [lab.js](#) and hosted on [Open Lab](#), both free tools for open, non-commercial research. Figure 1 shows the survey interface.

Figure 1



1.6 RESULTS

1.6.1 Participants

In total, 9 participants completed the survey. Participants had a median of 6 years of experience in their current role (range 1–10 years). Their job titles were data architect (3 participants), data engineer, data consultant, software architect, project manager, technical project manager, and PhD student. Six participants completed the survey in one sitting, taking on average 12 minutes (range 5–21 minutes). The remaining three participants left the survey open in between, which resulted in completion times between 3 hours and 2 days.

1.6.2 Example pipelines

The pipelines had between 3 and 10 steps (median 6). Four participants built a strictly linear pipeline. One of them deliberately split it into two separate flows, one for ingestion and one for transformation, so that the raw data is always available if the transformation needs to be rerun. The other five pipelines were not linear: four branched out into several following steps, and three had several steps merging into one. Branches were mostly found near the end of a pipeline, where data was written to several destinations (for example a database and an external API, or a triple store and an LDES), or where a data quality report led to different follow-up actions. Only one participant branched in the middle of the pipeline, enriching the data in two parallel steps before merging it again. Merges usually brought several inputs together into one step, such as data from different sources, or incoming data and the mapping files needed to transform it.

Most pipelines followed a familiar extract-transform-load pattern. Almost all started by reading data from a source, such as an API, a database, an HTTP endpoint or an LDES, and ended by writing data to storage, such as a database, triple store, lakehouse or blob storage. In between, participants transformed, combined and validated the data. Four pipelines included explicit validation or data quality checks, and three combined data from several sources. Three participants first stored an untouched copy of the raw data before processing it, two of them using a layered bronze-silver-gold setup. Three participants started by reading configuration or metadata, such as mapping files or a list of tables to process, which then determined what the rest of the pipeline did. Four pipelines were clearly built around Linked Data, using RML mappings, SPARQL, triple stores or LDES. Finally, the level of detail varied a lot: some participants described generic steps (one used a single "Process Data" step to stand for many steps), while others named the specific tools used in each step, such as dbt, PySpark notebooks or SPARQL Construct queries.

1.6.3 Step categories

To connect the pipelines to Part 1, we mapped each of the 52 steps to the 15 categories in Table 2, based on what the step's title and description said it does. Each step got one main category, and a second one if the description clearly contained a second action (for example, reading data from an API and

converting it to Linked Data). One step could not be categorized, because it described improving the pipeline itself rather than processing data. Table 4 shows how often each category occurred and where in the pipeline it typically appeared, from the first steps (early) to the last (late).

Table 4

parent category	category	number of steps	number of participants	typical position
In/Out	Read In Data	13	9	early
In/Out	Write Out Data	13	9	late
Refinement	Data Manipulation	12	7	middle
Refinement	Data Validation	5	4	middle
Refinement	Data Selection	5	3	middle
Pipeline Execution	Flow Control	4	3	late
In/Out	Data Representation	3	3	early
Insight	Data Visualization	2	2	late
In/Out	Communication	2	2	varies
Insight	Data Summarization	1	1	late
In/Out	File Handling	1	1	early

Read In Data and Write Out Data were the most common categories and appeared in every pipeline. Data Manipulation came next, appearing in seven of the nine pipelines, followed by Data Validation and Data Selection in three to four pipelines each. Looking at the parent categories, about half of all steps belonged to In/Out and 40% to Refinement, while Pipeline Execution and Insight steps were rare. Four categories did not appear at all: Data Inference, Data Learning, Execute Process and Data Generation. In other words, the pipelines participants build in their daily work are mainly about moving and preparing data. Analysing data or learning from it seems to happen outside these pipelines, if at all.

The linked steps showed a consistent order. Pipelines started by reading in data, often followed directly by converting it to another representation, such as mapping JSON data to RDF. The most common links were from Read In Data to Data Manipulation, and from Data Manipulation to either another manipulation step or Write Out Data. Data Selection and Data Validation sat in the middle of the pipeline. Validation happened either right after reading in the data, or after the data had been transformed and combined, just before writing it out. Flow Control steps came near the end, where data was split or fragmented across several destinations, or where metrics about the pipeline run were recorded. Pipelines ended by writing data to storage, sometimes followed by a step that visualizes or summarizes the result. In four pipelines, data was also written out earlier on, usually to keep a raw copy before further processing. Taken together, the most logical order of steps is:

Read In Data → Data Representation → Data Validation → Data Selection / Data Manipulation → Flow Control → Write Out Data → Data Visualization / Data Summarization

1.6.4 Pipeline setup and reuse

Participants estimated that setting up a pipeline for a project takes 7.9 days on average (median 5 days, range 2–20 days; 7 participants answered, 2 chose "don't know / not applicable"). On average, they estimated that 66% of their custom data processing logic can be reused across projects (median 70%, range 20–100%; 8 participants answered, 1 chose "don't know / not applicable"). When asked which parts of their pipelines cannot be reused, participants most often mentioned logic that is specific to a data source or use case, such as business logic, custom queries (e.g. GraphQL), mappings (e.g. RML) and data quality rules. Connecting to external sources was also described as hard to reuse, because every API or source, such as SFTP, works differently, for example in how it handles pagination. Other participants mentioned transformations tailored to specific front-end use cases, and configuration that depends on the transformed data. One participant noted that existing components could be reused, but that it was hard to find out what input and output they expect.

1.7 CONCLUSION

The survey showed that, while every participant built their pipeline in their own words and with their own tools, the pipelines themselves looked remarkably alike. Almost all followed an extract-transform-load pattern: read data from a source, transform, combine and validate it, and write it to storage. The steps also appeared in a consistent order across participants, and most of them fell into just a handful of the 15 categories from Part 1, mainly reading, writing and manipulating data. Where pipelines branched, it was usually at the end, to send the same data to several destinations. Recurring design choices, such as keeping a raw copy of the data or driving the pipeline from configuration and metadata, also came up in several pipelines.

At the same time, participants estimated that setting up a new pipeline still takes around a week, and that about a third of their custom logic cannot be reused in the next project. The parts that are hard to reuse are mostly specific to a data source or use case: business rules, queries and mappings, and connectors to APIs that all work slightly differently. One participant also pointed out that even when components can be reused, it is hard to find out what input and output they expect. With nine participants, these results give an indication rather than a representative picture, but they point in a clear direction: the overall shape of pipelines is highly repetitive, while the effort goes into the parts that differ.

GENERAL CONCLUSION

No single framework does everything. Part 1 showed that most frameworks are specialized. Frameworks built for Linked Data handle RDF well, but are usually tied to one programming language and often limited in how pipelines can branch or stream. General-purpose frameworks such as Apache NiFi and KNIME offer large libraries of components, but need extensions or custom work to process RDF. Even RDF-Connect, which offered all the capabilities we assessed, has a much smaller library of ready-made steps than the general-purpose frameworks. In practice, this means that teams either pick one framework and work around its gaps, or write custom code, which is exactly what makes pipelines slow to build and hard to reuse.

The Generic Toolchain Architecture is designed to address this in two ways. First, it makes frameworks interoperable. A pipeline is written once as a framework-independent pipeline definition in RDF, following the semantic model of the Generic Toolchain Architecture. The pipeline generator looks up the required components in a shared catalog, generates the configuration files for each framework, connects steps that run in different frameworks automatically, and wires everything together in a single Docker Compose setup. It currently supports RDF-Connect, LDIO, Apache NiFi and semantic.works. This lets one pipeline combine the strengths of several frameworks, for example the streaming and RDF capabilities of RDF-Connect with the large component library of Apache NiFi, instead of being limited by the weaknesses of one.

Second, the Generic Toolchain Architecture can make use of the common pipeline patterns found in Part 2. Because pipeline definitions are plain, semantic descriptions, a pipeline that follows a common pattern can be shared, edited and reused as a starting point for the next project, instead of being rebuilt from scratch. In the same way, commonly used transformation steps are described once as components in the catalog and can be reused in any pipeline. The catalog also describes the configuration each component expects and validates pipeline definitions against it before anything is generated, which addresses the difficulty of knowing what input and output a component expects. Source-specific logic will still need custom work, but by reusing both pipeline patterns and components, teams only have to build the parts that are truly specific to their project.